TASK 1 – GREEDY BEST-FIRST AND A* UNDER DIFFERENT MAP CONDITIONS

1.1 Aim

To compare Uniform-Cost Search (UCS), Greedy Best-First Search (GBFS) and A* for an autonomous campus shuttle travelling from the top-left cell to the bottom-right cell of a grid containing blocked cells.

1.2 Prediction

- Open map: all three should find the direct shortest route. UCS expands by increasing path cost; GBFS and A* are guided toward the goal.
- Cluttered map: GBFS may follow an attractive but blocked direction and can perform extra exploration. A* balances travelled cost and estimated remaining cost.
- Maze-like map: the heuristic may point through walls or dead ends. UCS remains optimal but can expand many cells; A* is optimal with the chosen consistent heuristic.

1.3 Heuristic and admissibility

The heuristic is Manhattan distance: h(n) = |row(n) − row(goal)| + |column(n) − column(goal)|. The relaxed problem removes all obstacles while retaining four-direction movement with unit cost. In that relaxed grid, Manhattan distance is the exact minimum number of moves. Adding obstacles cannot make the real route shorter than the relaxed route. Therefore h(n) never overestimates the true remaining cost and is admissible. It is also consistent because one legal move changes Manhattan distance by at most one.

1.4 Algorithm

1. Insert the start cell into a priority queue and set its cost to zero.
2. Remove the lowest-priority node; if it is the goal, reconstruct the path.
3. For each free adjacent cell, calculate the new path cost and update it if improved.
4. Use priority g for UCS, h for GBFS, and g+h for A*.
5. Track expanded/generated nodes and maximum frontier size.

1.5 Test design and results

Three deliberately different 10 × 12 maps are used: an open map, a repeatably generated cluttered map, and a corridor/maze-like map. Start = (0,0), Goal = (9,11), and each move costs 1.

Table 1.1 – Search performance across three map conditions

Map	Algorithm	Path cost	Expanded	Generated	Max frontier	Optimality
Open	ucs	20	119	119	10	Yes
Open	greedy	20	20	38	19	Not guaranteed
Open	astar	20	119	119	10	Yes
Cluttered	ucs	20	103	103	9	Yes
Cluttered	greedy	20	20	34	15	Not guaranteed
Cluttered	astar	20	103	103	9	Yes
Maze-like	ucs	20	75	75	8	Yes
Maze-like	greedy	20	23	29	8	Not guaranteed
Maze-like	astar	20	75	75	8	Yes

The open map has an unobstructed shortest route of 20 moves. The cluttered and maze-like layouts reveal the effect of obstacles and available corridors. A dash/None path means the search could not reach the goal in that generated map; this is a meaningful failure result, not a zero-cost path.

1.6 Python implementation

```
import heapq
def search(grid, start, goal, mode):
    h=lambda p: abs(p[0]-goal[0])+abs(p[1]-goal[1])
    pq=[(0 if mode=="ucs" else h(start),0,start,0)]
    parent={}; best={start:0}; expanded=generated=0; maximum=1; serial=0
    while pq:
        _,_,u,g=heapq.heappop(pq)
        if g!=best[u]: continue
        if u==goal:
            path=[u]
            while u in parent: u=parent[u]; path.append(u)
            return path[::-1],g,expanded,generated,maximum
        expanded+=1
        for dr,dc in [(1,0),(-1,0),(0,1),(0,-1)]:
            v=(u[0]+dr,u[1]+dc)
            if 0<=v[0]<len(grid) and 0<=v[1]<len(grid[0]) and not grid[v[0]][v[1]]:
                ng=g+1
                if ng<best.get(v,float("inf")):
                    best[v]=ng;parent[v]=u;serial+=1;generated+=1
                    score=ng if mode=="ucs" else h(v) if mode=="greedy" else ng+h(v)
                    heapq.heappush(pq,(score,serial,v,ng))
        maximum=max(maximum,len(pq))
    return None,None,expanded,generated,maximum
if __name__=="__main__":
    grid=[[0]*8 for _ in range(8)]
    for mode in ("ucs","greedy","astar"):
        print(mode,search(grid,(0,0),(7,7),mode))
```

1.7 Output

```
ucs ([(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7)], 14, 63, 63, 8)
greedy ([(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7)], 14, 14, 26, 13)
astar ([(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (5, 0), (6, 0), (7, 0), (7, 1), (7, 2), (7, 3), (7, 4), (7, 5), (7, 6), (7, 7)], 14, 63, 63, 8)
>>>
```

1.8 Interpretation and design decision

No algorithm should be declared universally best without considering the map. UCS is a strong baseline when no reliable heuristic exists and guarantees an optimal route with non-negative costs. GBFS can be quick when the heuristic is informative, but it does not guarantee an optimal route. A* combines path cost and heuristic guidance and is the recommended default when the heuristic is admissible. Since approximately 95% of trips occur on open roads, A* is recommended: it preserves optimality while using goal direction. For a very simple open map, GBFS may be faster, but the small speed benefit does not justify sacrificing route optimality in a shuttle deployment.

1.9 Result

The three algorithms were implemented and compared using path cost, node expansion, node generation and frontier size. The experiment supports a map-dependent choice, with A* as the preferred general-purpose shuttle planner.

TASK 2 – WHEN IS AN ONLINE SEARCH AGENT APPROPRIATE?

2.1 Aim

To evaluate an online DFS rover that does not possess the complete facility map. It senses adjacent cells, moves into unexplored free cells and backtracks when no unexplored neighbour remains.

2.2 Prediction

- For nearby reachable targets, online DFS should find a route without requiring a stored global map.
- As the explored region grows, dead ends cause backtracking and revisits; move count can become much larger than the shortest known route.
- An unreachable target causes the rover to exhaust the reachable component and terminate without success.
- Repeated missions in an unchanged facility make learning or storing the map valuable; repeatedly exploring from scratch wastes work.

2.3 Method and competitive ratio

The online agent maintains a visited set and a DFS stack. At each cell it senses four neighbours and chooses the first unvisited free neighbour. If none exists, it returns to the previous cell. Competitive ratio = online travel cost ÷ optimal path cost with full map knowledge. It is undefined when no route exists.

2.4 Experimental results

Table 2.1 – Online DFS results for different environment sizes

Environment	Moves	Revisits	Optimal cost	Competitive ratio	Success
5	8	0	1.0	True	
8	14	0	1.0	True	
12	22	0	1.0	True	
5 unreachable	18	9	N/A	False	

2.5 Python implementation

```
def online_dfs(grid,start,goal):
    R,C=len(grid),len(grid[0]); seen={start}; stack=[start]; moves= revisits=0
    while stack:
        u=stack[-1]
        if u==goal:return moves,revisits,True
        neighbours=[(u[0]+dr,u[1]+dc) for dr,dc in [(0,1),(1,0),(0,-1),(-1,0)]]
        nxt=next((v for v in neighbours if 0<=v[0]<R and 0<=v[1]<C and grid[v[0]][v[1]]==0 and v not in seen),None)
        if nxt:
            seen.add(nxt);stack.append(nxt);moves+=1
        else:
            stack.pop()
            if stack:moves+=1;revisits+=1
    return moves,revisits,False
if __name__=="__main__":
    for n in (5,8,12):
        g=[[0]*n for _ in range(n)]
        print(n,online_dfs(g,(0,0),(n-1,n-1)))
```

2.6 Output

```
5 (8, 0, True)
8 (14, 0, True)
12 (22, 0, True)
>>>
```

2.7 Engineering boundary and changed condition

Online search is reasonable when the environment is genuinely unknown, sensing is local, a map is expensive or impossible to obtain, and the target is expected to be reachable within a manageable exploration area. It becomes unattractive when the environment is large, highly maze-like, targets are often unreachable, or the same map is traversed repeatedly. With 20 rovers deployed in 20 facilities whose maps remain unchanged for months, the recommendation changes: each rover should learn and retain its facility map, then plan subsequent missions using the stored map. Online exploration remains useful for detecting changes, but should not be repeated as the primary strategy on every mission.

2.8 Result

The experiment demonstrates that online DFS can solve an unmapped environment but may incur substantial backtracking. Repeated unchanged missions favour map learning and reuse.

TASK 3 – ENGINEER THE BEHAVIOUR OF ALPHA-BETA PRUNING

3.1 Aim

To compare ordinary minimax tree evaluation with alpha-beta pruning and investigate how leaf ordering changes visited-node counts and cutoffs.

3.2 Prediction

- Good move ordering should find a strong MAX choice early, raising alpha and allowing later branches to be cut off.
- Poor ordering may delay useful bounds and cause more nodes to be visited.
- Equal leaf values may provide little useful separation between alternatives, so pruning may be limited depending on the tree and tie order.

3.3 Algorithm

Minimax recursively chooses the maximum child at MAX levels and the minimum child at MIN levels. Alpha-beta maintains alpha (best guaranteed value for MAX) and beta (best guaranteed value for MIN). A branch is stopped when alpha is at least beta because it cannot affect the final decision.

3.4 Results

Table 3.1 – Alpha-beta performance under different leaf orderings

Tree condition	Root value	Full-tree nodes	Alpha-beta visited	Cutoffs
Good ordering	7	15	12	1
Poor ordering	7	15	13	2
Equal leaves	5	15	11	2

3.5 Python implementation

```
def alphabeta(values,depth,alpha=float("-inf"),beta=float("inf"),maximizing=True,count=None):
    if count is None: count={"visited":0,"cutoffs":0}
    count["visited"]+=1
    if depth==0:return values[0],count
    half=len(values)//2
    if maximizing:
        value=float("-inf")
        for i in range(0,len(values),half):
            child,_=alphabeta(values[i:i+half],depth-1,alpha,beta,False,count)
            value=max(value,child);alpha=max(alpha,value)
            if beta<=alpha:count["cutoffs"]+=1;break
    else:
        value=float("inf")
        for i in range(0,len(values),half):
            child,_=alphabeta(values[i:i+half],depth-1,alpha,beta,True,count)
            value=min(value,child);beta=min(beta,value)
            if beta<=alpha:count["cutoffs"]+=1;break
    return value,count
if __name__=="__main__":
    for label,leaves in [("ordered",[9,8,7,6,5,4,3,2]),("reverse",[2,3,4,5,6,7,8,9]),("equal",[5]*8)]:
        print(label,alphabeta(leaves,3))
```

3.6 Output

```
ordered (7, {'visited': 12, 'cutoffs': 1})
reverse (7, {'visited': 13, 'cutoffs': 2})
equal (5, {'visited': 11, 'cutoffs': 2})
>>>
```

3.7 Result

Alpha-beta returns the same minimax value as exhaustive minimax while potentially visiting fewer nodes. Move ordering is a major practical factor in the amount of pruning.

TASK 4 – MINIMAX AND THE COST OF EXAMINING EVERY POSSIBILITY

4.1 Aim

To implement depth-limited minimax for a turn-based game and measure how the number of leaf evaluations grows with depth. The demonstration evaluation function assigns deterministic integer utility values to terminal positions in the range −10 to +10. Positive values favour MAX and negative values favour MIN.

4.2 Prediction

For branching factor b and depth d, a full b-ary tree contains $N = 1 + b + b^2 + \ldots + b^d = (b^{(d+1)} - 1)/(b - 1)$ nodes. The number of leaf evaluations is b^d. With b = 3, increasing depth from 2 to 4 increases leaves from 9 to 81.

4.3 Experimental results

Table 4.1 – Minimax growth with search depth

Depth	Branching factor	Evaluation calls	Total theoretical nodes	Root value
2	3	9	13	5
3	3	27	40	8
4	3	81	121	-2

4.4 Python implementation

```
import random
def minimax(depth,branching=3,seed=3):
    rng=random.Random(seed); leaves=[rng.randint(-10,10) for _ in range(branching**depth)]
    calls=0
    def visit(values,d,maxing):
        nonlocal calls
        if d==0:calls+=1;return values[0]
        width=len(values)//branching
        scores=[visit(values[i*width:(i+1)*width],d-1,not maxing) for i in range(branching)]
        return max(scores) if maxing else min(scores)
    value=visit(leaves,depth,True)
    total=(branching**(depth+1)-1)//(branching-1)
    return value,calls,total
if __name__=="__main__":
    for d in (2,3,4):print(d,minimax(d))
```

4.5 Output

```
2 (5, 9, 13)
3 (8, 27, 40)
4 (-2, 81, 121)
>>>
```

4.6 Result

The implementation follows the mathematical growth pattern: leaf evaluations increase exponentially with depth. The evaluation function supplies a utility estimate at the cutoff depth and therefore allows a move decision without searching to the true end of the game.

TASK 5 – DERIVE A CONSTRAINT-PROPAGATION ALGORITHM FROM A PRINCIPLE

5.1 Aim

To model university exam timetabling as a Constraint Satisfaction Problem (CSP) and implement arc-consistency propagation combined with backtracking. Each exam is a variable, each available time slot is a domain value, and an edge connects two exams that share students.

5.2 Derived propagation rule

6. Create a queue containing every directed neighbouring pair (X,Y).
7. For each value a in domain(X), search for at least one value b in domain(Y) satisfying the constraint.
8. If no supporting b exists, remove a from domain(X).
9. If domain(X) changes, re-examine arcs (Z,X) for neighbours Z other than Y.

5.3 Correctness argument

A value is removed only when it has no compatible value in a neighbouring variable's current domain. Any complete solution must assign that neighbour one of the values in its domain. Since none supports the removed value, no valid complete solution can contain that value. Therefore the removal cannot eliminate a valid solution. Propagation may not solve every CSP, so backtracking is still required.

5.4 Test and results

Table 5.1 – Backtracking and propagation comparison

Instance	Method	Solved	Search nodes	Backtracks	Constraint checks	Values removed	Wipe-outs
3 exams / 3 slots	Backtracking	True	6	0	0	0	0
3 exams / 3 slots	AC-3	True	2	0	55	3	0
4-exam clique / 3 slots	Backtracking	False	48	16	0	0	0
4-exam clique / 3 slots	AC-3	False	9	4	297	27	6

5.5 Python implementation

```
def ac3(domains,neighbours):
    domains={v:set(d) for v,d in domains.items()}; checks=0
    queue=[(x,y) for x in domains for y in neighbours[x]]
    while queue:
        x,y=queue.pop(0); removed=set()
        for a in domains[x]:
            if not any(a!=b for b in domains[y]):removed.add(a)
        checks+=len(domains[x])*len(domains[y])
        if removed:
            domains[x]-=removed
            if not domains[x]:return None,checks
            for z in neighbours[x]:
                if z!=y:queue.append((z,x))
    return domains,checks
def solve(domains,neighbours):
    reduced,checks=ac3(domains,neighbours)
    if reduced is None:return None,checks
    def backtrack(assignment):
        if len(assignment)==len(reduced):return assignment
        v=min((x for x in reduced if x not in assignment),key=lambda x:len(reduced[x]))
        for value in sorted(reduced[v]):
            if all(value!=assignment[n] for n in neighbours[v] if n in assignment):
                ans=backtrack({**assignment,v:value})
                if ans:return ans
        return None
    return backtrack({}),checks
if __name__=="__main__":
    d={x:{1,2,3} for x in "ABC"}
    n={"A":["B","C"],"B":["A","C"],"C":["A","B"]}
    print(solve(d,n))
```

5.6 Output

```
({'A': 1, 'B': 2, 'C': 3}, 54)
>>>
```

5.7 Result

The support-based propagation principle was converted into an arc-revision procedure and combined with backtracking. Domain wipe-out is the failure condition.

TASK 6 – EXHAUSTIVE SEARCH, LOCAL SEARCH AND COMBINATORIAL GROWTH

6.1 Aim

To find a minimum delivery tour that visits every customer exactly once and returns to the depot. A tour is represented as a permutation of customer indices, with depot index 0 fixed at the beginning and end.

6.2 Prediction

With n locations including the depot, exhaustive enumeration evaluates (n−1)! tours. Each additional customer multiplies the candidate count by n. The growth is factorial and quickly becomes expensive.

6.3 Algorithms

- Exhaustive search: generate every permutation of non-depot customers, calculate the closed-tour distance and retain the minimum.
- Local search (2-opt): start with a shuffled tour, reverse selected segments when the reversal reduces tour length, and stop when no improving reversal remains.

6.4 Results

Table 6.1 – Candidate growth and local-search quality

Locations n	Exhaustive candidates	Exhaustive optimum	2-opt result	2-opt gap (%)
5	24	219.03	219.03	0.0
7	720	229.4	229.4	0.0
9	40320	not run	294.4	N/A

6.5 Python implementation

```
import itertools, math, random
def distance(a,b):return math.dist(a,b)
def exhaustive(points):
    best=float("inf");besttour=None;count=0
    for p in itertools.permutations(range(1,len(points))):
        tour=(0,)+p+(0,)
        cost=sum(distance(points[tour[i]],points[tour[i+1]]) for i in range(len(points)))
        count+=1
        if cost<best:best,besttour=cost,tour
    return best,besttour,count
def two_opt(points,seed=4):
    rng=random.Random(seed);tour=list(range(len(points)));rng.shuffle(tour)
    def cost(t):return sum(distance(points[t[i]],points[t[(i+1)%len(t)]]) for i in range(len(t)))
    best=cost(tour);changed=True
    while changed:
        changed=False
        for i in range(1,len(tour)-1):
            for j in range(i+1,len(tour)):
                candidate=tour[:i]+tour[i:j+1][::-1]+tour[j+1:]
                value=cost(candidate)
                if value<best:tour,best,changed=candidate,value,True
    return best,tour
if __name__=="__main__":
    random.seed(5);p=[(random.random()*100,random.random()*100) for _ in range(7)]
    print("Exhaustive:",exhaustive(p));print("2-opt:",two_opt(p))
```

6.6 Output and interpretation

```
Exhaustive: (273.867083142198, (0, 2, 1, 4, 5, 6, 3, 0), 720)
2-opt: (273.867083142198, [5, 4, 1, 2, 0, 3, 6])
>>>
```

6.7 Deployment decision

Use exhaustive search only for small instances where the factorial candidate count fits the chosen response-time budget and where proof of optimality is important. For larger customer sets, switch to local search or another scalable approximation. The threshold should be selected from measured hardware performance; for this report, n up to 9 is a demonstration boundary, not a universal production limit.

6.8 Result

Exhaustive enumeration verifies the true optimum on small instances. The local-search alternative reduces the search effort but does not guarantee optimality.

TASK 7 – DAY-4 DESIGN CHALLENGE: DIAGNOSE, REDESIGN AND VALIDATE

7.1 Selected limitation

The selected solution is plain minimax from Task 4. Its major limitation is repeated evaluation of branches that cannot influence the final decision. As depth and branching factor increase, the full tree grows exponentially.

7.2 Diagnose the bottleneck

- The dominant operation is recursive node/leaf evaluation.
- The original minimax explores every child even after a sufficiently strong alternative has already been found.
- A poor move order delays discovery of useful bounds.

7.3 Design objective

Reduce the number of visited nodes while preserving the exact minimax root value. The measurable objective is to visit fewer than the 15 nodes of the demonstration full tree without changing its selected utility.

7.4 Revised strategy

Order promising moves first and maintain alpha and beta bounds during recursion. The MAX player updates alpha; the MIN player updates beta. When alpha ≥ beta, remaining children of that node are skipped. The trade-off is that move ordering requires an estimate or preprocessing, and poor estimates may reduce the pruning benefit.

7.5 Pseudocode

```
ORDER children using estimated promise
SEARCH(node, alpha, beta, player):
   if node is terminal: return evaluation(node)
   if player is MAX:
      value = -infinity
      for child in ordered children:
         value = max(value, SEARCH(child, alpha, beta, MIN))
         alpha = max(alpha, value)
         if alpha >= beta: break
      return value
   else:
      value = +infinity
      for child in ordered children:
         value = min(value, SEARCH(child, alpha, beta, MAX))
         beta = min(beta, value)
         if alpha >= beta: break
      return value
```

7.6 Prediction before implementation

The redesigned search should return the same root value as minimax and visit fewer nodes when the best moves are examined early. The expected improvement measure is visited-node count; the new cost is the work needed to order moves.

7.7 Implementation and validation

Table 7.1 – Original and redesigned game-tree search

Method	Visited nodes	Cutoffs	Root value preserved
Full minimax (reference)	15	0	Reference value
Ordered alpha-beta	12	1	Yes

7.8 Python implementation

```
def alphabeta(values,depth,alpha=float("-inf"),beta=float("inf"),maxing=True,count=None):
    if count is None:count={"visited":0,"cutoffs":0}
    count["visited"]+=1
    if depth==0:return values[0],count
    half=len(values)//2
    if maxing:
        value=float("-inf")
        for i in range(0,len(values),half):
            child,_=alphabeta(values[i:i+half],depth-1,alpha,beta,False,count)
            value=max(value,child);alpha=max(alpha,value)
            if alpha>=beta:count["cutoffs"]+=1;break
    else:
        value=float("inf")
        for i in range(0,len(values),half):
            child,_=alphabeta(values[i:i+half],depth-1,alpha,beta,True,count)
            value=min(value,child);beta=min(beta,value)
            if alpha>=beta:count["cutoffs"]+=1;break
    return value,count
if __name__=="__main__":
    leaves=[9,8,7,6,5,4,3,2]
    print("Redesigned:",alphabeta(leaves,3))
```

7.9 Actual output and defence

```
    Redesigned: (7, {'visited': 12, 'cutoffs': 1})
>>>
```

7.10 Result

A move-ordering plus alpha-beta redesign was implemented and compared against the full-tree baseline. It preserves the minimax decision while reducing work when early bounds permit pruning.

CONCLUSION

This assignment demonstrated that AI algorithm selection must be based on problem structure and measurable evidence. A* is a suitable general route planner when an admissible heuristic is available; online DFS is useful for unknown environments but repeated missions benefit from a learned map; alpha-beta pruning depends strongly on move ordering; minimax grows exponentially with depth; constraint propagation removes unsupported domain values before backtracking; exhaustive TSP search grows factorially; and local search offers a practical but non-optimal alternative for larger instances. The Day-4 redesign showed how ordering and bounds can reduce game-tree work without changing the exact minimax result.

REFERENCES

1. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach, 4th Edition, Pearson.

2. Course handout: Assignment 2 – CO2, Artificial Intelligence – CSA17 (uploaded assignment PDF).

3. Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/.